Inverting Python: Designing NTP-Prior-Resistant Programming Languages for LLM-Proof Coding Assessment

Jaeyeong CHOI
DGIST EECS
jaeyeong2022@dgist.ac.kr

***Abstract*—LLMs now solve standard programming tasks well enough to undermine their value as student assessments. We investigate whether it is possible to design a programming language that is human-interpretable but defeats LLM next-token-prediction (NTP) priors. We propose a four-level taxonomy—L1 (token alias substitution), L2 (syntax inversion), L3 (semantic inversion with explicit rules), L4 (semantic inversion with implicit delivery)—and evaluate LLM compliance across five model families. L1 creates partial resistance (KLR 0.26–0.69); L2 is easily adopted; L3 reveals a model capability threshold; and L4 defeats most tested model-task combinations through *pattern blindness*. A multi-task extension (Fibonacci, is_sorted, count_vowels) reveals task-dependent resistance and a third failure mode, *operational substitution*. We identify three failure mechanisms—*prior dominance*, *pattern blindness*, and *operational substitution*—and show that L4-style language design provides robust resistance for programming education.**

***Index Terms*—confusion language, LLM resistance, NTP prior, semantic inversion, programming education, code assessment**

I. Introduction

Modern code-generation benchmarks [1], [2] show that large language models (LLMs) can solve standard programming tasks with high accuracy. This creates a critical challenge for programming education: if LLMs solve assignments faster and more accurately than students, how can instructors meaningfully assess student understanding?

This paper studies a complementary but understudied question:

> *Is it possible to design a programming language that is human-interpretable but systematically defeats LLM next-token-prediction priors—making it resistant to AI-based code generation?*

We decompose this into three sub-questions:

1) Does changing token identifiers (L1: alias substitution) create effective LLM resistance?
2) Does inverting syntactic structure (L2) create effective resistance?
3) Does inverting semantic behavior (L3/L4) create deeper resistance?

Our key empirical findings are:

- **L1 (token substitution):** Partially effective—keyword leakage rate (KLR) ranges from 0.26 to 0.69 across models. Models revert to Python priors at measurable rates.
- **L2 (syntax inversion):** Not effective—models adopt new syntax completely (SIR=0.0) when given explicit rules.
- **L3 (semantic inversion, explicit):** Capacity-dependent—weaker models fail completely (PPR=1.0 for gpt-4o-mini), while stronger models succeed (PPR=0.0 for gpt-4o).
- **L4 (semantic inversion, implicit):** Effective against all tested models—even strong models exhibit *pattern blindness*, ignoring semantic inversion cues in examples.

We also identify three distinct LLM failure modes: *prior dominance* (prior overrides stated rules), *pattern blindness* (model fails to extract semantic rules from examples), and *operational substitution* (model achieves correct output via arithmetic workaround rather than adopting the semantic rule—observed on count_vowels for gpt-4o).

This framework provides both a theoretical taxonomy for LLM-resistant language design and practical tools for constructing programming assessments that remain meaningful in the LLM era.

II. Related Work

A. Code Generation Benchmarks and Evaluation

HumanEval [1] and MBPP [2] established functional correctness as the standard metric for code-generation evaluation. More recent benchmarks scale and diversify tasks: BigCodeBench [3] adds complex, multi-function instructions, and LiveCodeBench [4] avoids contamination through continuous updating. Across all these benchmarks, Python dominates training corpora, embedding strong token-level priors into all models evaluated. This motivates our L1 baseline: even with explicit alias instructions, models leak Python keywords at 26–69%.

B. Code-Oriented LLM Architectures and Training

Pre-trained code models—CodeBERT [5], CodeT5 [6], and InCoder [7]—demonstrate that NTP objectives on Python code reinforce keyword position priors. More recent instruction-tuned models (WizardCoder [8], DeepSeek-Coder [9]) amplify these priors through even larger code corpora. This trajectory predicts our finding: *stronger code LLMs do not necessarily mean better alias compliance*—gpt-5.4-mini (KLR=0.67) underperforms gpt-4o (KLR=0.37).

C. Instruction-Following in Code (2024–2025)

CodeIF [10] and MultiCodeIF [11] introduce benchmarks measuring LLM adherence to developer-specified constraints in code generation tasks. Critically, MultiCodeIF reports that models perform relatively well on *explicit, concrete* constraints but degrade substantially on *implicit or abstract* constraints, and that performance drops sharply as the number of hierarchical constraints increases. This explicit–implicit asymmetry is the closest mechanistic evidence in the literature to our L3/L4 findings: models can follow explicit semantic inversion rules (L3, with some model-capacity dependence), yet fail entirely when the same semantic inversion is delivered only through examples without explicit articulation (L4, pattern blindness across all tested models). The key distinction is scope: CodeIF and MultiCodeIF benchmark *constraint-following* within standard programming tasks, whereas our work *designs* a programming language whose semantic structure systematically conflicts with NTP priors, making resistance a structural property rather than a task-level benchmark artifact.

D. Instruction Hierarchy and Prior Conflicts

Chen et al. [12] propose an instruction hierarchy to resolve conflicts between system-level and user-level instructions. Their framework addresses natural-language directive conflicts; ours involves programming-language semantic rule conflicts—a dimension not studied in alignment work. Xie et al. [13] and Wu et al. [14] show that when context conflicts with parametric knowledge, LLMs often default to pretraining priors. Zhang et al. [15] survey this knowledge conflict problem broadly. Shi et al. [16] demonstrate LLMs are easily distracted by irrelevant context; our L4 condition represents the inverse: models *fail to use* relevant semantic cues provided in examples.

E. NTP Priors and In-Context Learning

Wang et al. [17] formalize a "law of next-token prediction" providing theoretical grounding for how NTP training shapes output distributions. Min et al. [18] show ICL demonstrations affect outputs primarily through distributional surface patterns rather than semantic content—explaining why models fail to extract inverted semantics from L4 examples. Dong et al. [19] survey ICL comprehensively. Lampinen et al. [20] document "A-Not-B" errors where LLMs fail to suppress previously established response patterns under context changes, directly analogous to our L4 pattern blindness.

F. Counterfactual and Semantic Reasoning Failures

Zhao et al. [21] show LLMs struggle when counterfactual reasoning requires overriding parametric knowledge—the same mechanism we observe at L3 for weak models. Turpin et al. [22] show LLM self-explanations are often unfaithful to actual behavior. Valmeekam et al. [23] document reasoning rigidity—models revert to memorized strategies under redefined rules—providing further precedent for our pattern blindness finding.

G. Adversarial Perturbation and Code Resistance

PromptBench [24] demonstrates LLM fragility under text perturbations. Pan et al. [25] show semantically aligned code prompt perturbations cause up to 42% accuracy drops. Jha et al. [26] use structured adversarial prompts to break LLM reasoning in code tasks. Khan et al. [27] specifically design prompt perturbations to reduce LLM correctness on programming assignments by 77%, most directly related to our goal. Our approach is complementary: instead of perturbing prompts, we design a confusion language whose NTP prior conflict is a *structural property*, making it principled and reproducible.

H. DSL and Constrained Code Generation

Wu et al. [28] survey LLM performance on low-resource and domain-specific languages, confirming that Python-trained models underperform on less common languages. ConCodeEval [29] benchmarks code constraint comprehension in DSLs; L2CEval [30] evaluates language-to-code generation across specialized languages. Constrained decoding [31], [32] enforces grammar-level output validity. Our L2 experiment shows models adopt explicit syntax rules entirely, but semantic constraints (L3/L4) prove far more resistant—showing grammar-level constraints are insufficient without semantic-level challenges.

I. Real-World Engineering Benchmarks

SWE-bench [33] reveals that LLMs fail to generalize from canonical benchmarks to real repository-level tasks—confirming that high performance on standard benchmarks does not indicate robustness to structural novelty, as we observe in our L1 L4 evaluations.

J. LLM in Programming Education

Dakhel et al. [34] show LLM coding tools are an "asset or liability" depending on student expertise. Lau and Guo [35] document novice programmers adopting LLM outputs uncritically. Prather et al. [36] show mixed student perceptions. Liffiton et al. [37] propose guardrailed LLM tutoring. These studies confirm a shared concern: LLM integration can undermine independent skill development.

K. Academic Integrity and LLM-Resistant Assessment

Novak and McDanel [38] propose contextual, personalized assignments to resist LLM cheating—the most directly related paper to our educational goal—but without a formal theory of what features drive LLM failure. Code perplexity detection [39] is brittle; our taxonomy offers a principled design-based solution. RLHF-based alignment [40] improves instruction-following but, as our L3/L4 results show, remains insufficient when semantic inversions must be extracted from examples.

L. Research Gap

Prior adversarial work degrades generation quality without principled language design. Prior DSL work measures performance without explaining failure mechanisms. Prior knowledge-conflict work focuses on factual knowledge, not programming-language semantics. Our contribution closes these gaps: a

TABLE I
FOUR-LEVEL CONFUSION LANGUAGE TAXONOMY. EACH LEVEL TARGETS A DISTINCT LINGUISTIC DIMENSION AND DELIVERY MODALITY. L4 IS THE ONLY LEVEL WHERE ALL TESTED MODELS FAIL REGARDLESS OF CAPACITY.

Level	Perturbation	Delivery	Observed failure mode
L1	Token alias	Explicit	Prior dominance (partial)
L2	Syntax inv.	Explicit	None—adopted
L3	Semantic inv.	Explicit	Capacity-dependent
L4	Semantic inv.	Implicit	Pattern blindness

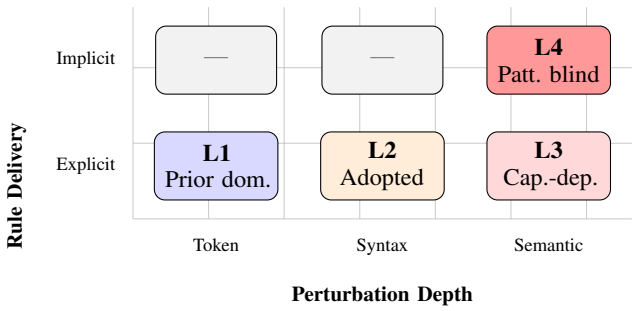

Fig. 1. Two-axis confusion language design space. Perturbation depth (token/syntax/semantic) and rule delivery (explicit/implicit) together determine LLM failure mode. L4 (dark red) occupies the hardest cell: semantic perturbation without explicit rule—the only condition where all tested models fail.

theory-grounded L1–L4 taxonomy that characterizes LLM failure by mechanism (prior dominance vs. pattern blindness) and guides principled LLM-resistant language design.

III. METHOD

A. Research Questions and Hypotheses

RQ1: At what language design level does LLM resistance become robust? **RQ2**: What failure mechanism is active at each level—prior dominance or pattern blindness?

H1: Token-level alias substitution (L1) creates partial but not complete resistance. **H2**: Syntactic inversion (L2) is insufficient—models adopt new syntax when given explicit rules. **H3**: Semantic inversion (L3/L4) provides robust resistance, but differs in mechanism by model capacity.

B. Language Level Taxonomy

We design a four-level taxonomy along two axes: (1) *perturbation depth* (token, syntax, semantic) and (2) *rule delivery* (explicit vs. implicit). Table I summarizes each level.

Figure 1 illustrates the two-axis design space and the location of each level.

C. L1: Token Alias Substitution

Python keywords (`if`, `for`, `in`, `return`, `def`) are replaced with aliases. Constraints: one-to-one mapping, no alias collision, token-boundary replacement, literal/comment protection.

Python (standard)

```
def fib(n):
if n <= 1:
return n
return fib(n-1) + fib(n-2)
```

Standard semantics: if-block runs when condition is TRUE

L3 (semantic inversion)

```
def fib(n):
if n <= 1:
return fib(n-1) + fib(n-2)
return n
```

Inverted: if-block runs when condition is FALSE

Fig. 2. Semantic inversion in L3. Surface syntax is identical to Python, but if-block semantics are inverted. The correct L3 solution (right) looks counter-intuitive to Python-trained models, which generate the left version regardless of the stated rule.

D. L2: Syntax Structure Inversion

Function definitions, conditionals, loops, and block delimiters are structurally inverted relative to Python. Example: `def fib(n):` becomes `:define fib [n] ->`. Model is given explicit grammar rules.

E. L3: Semantic Inversion (Explicit Rules)

Python surface syntax is retained but the semantics of if-blocks are inverted: the indented block executes when the condition is *false*. Model is explicitly told this rule. Figure 2 illustrates the concept.

F. L4: Semantic Inversion (Implicit, Example-Only)

Same as L3, but no rule is stated. Instead, models receive 2–3 worked examples that demonstrate the inversion. Models must extract the semantic pattern through few-shot reasoning.

G. Evaluation Metrics

Redundant metrics are removed. We report two independent measures:

- **KLR** (Keyword Leakage Rate): fraction of alias slots where original Python keyword was used. Note: ACR $= 1 -$ KLR and PIR $\approx$ KLR are derived quantities and not separately reported.
- **PSS** (Partial Structural Score): weighted sub-component credit (alias: 0.4, parse: 0.25, skeleton: 0.25, format: 0.1).
- **PPR** (Python Prior Rate, L3/L4): fraction of samples using Python-style condition (no inversion).

H. Threats to Validity

Internal: judge rules may be stricter than human acceptance; PSS weights are heuristic. **Construct**: single benchmark task (fib) limits generalization; multi-task extension is needed.

External: results cover five GPT-family models; open-model replication is future work.

IV. Experiments

A. Experimental Overview

We ran experiments at each language level for available model families. All runs used prompts from `v1-v120` (L1) or dedicated level-specific banks (L2–L4). The AVRTG key path was used for authenticated API calls.

B. Models Evaluated

- `gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`, `o4-mini`

C. L1 Experiment: Token Alias Substitution

120 prompts per model, versioned v1–v120 with varied alias mappings. Context-pack ablation: baseline vs. explicit alias contract system prompt.

D. L2 Experiment: Syntax Inversion

5 prompts using `:define ... ->` syntax (see Appendix). Model given complete grammar specification.

E. L3 Experiment: Semantic Inversion (Explicit)

10 prompts. System message explicitly states: "if-blocks execute when condition is FALSE." Evaluated on whether model inverted condition logic.

F. L4 Experiment: Semantic Inversion (Implicit)

Initial screening: 10 prompts per model, no rule stated; only 2 worked examples with inverted semantics shown. Model must infer pattern. Ablation (n=50): 5 prompt variants $\times$ 10 runs each, varying the number of worked examples (2 vs. 3), presence of inline comments, order of examples, and domain of examples. Evaluated across four models at $n = 50$: `gpt-4o`, `gpt-4o-mini`, `gpt-4.1-mini`, and `gpt-5.4-mini` (200 total runs). Extended ablation ($n = 20$, 5 variants $\times$ 4 runs) for `gpt-4.1`, `gpt-4.1-nano`, and `o4-mini` (60 additional runs).

G. L4 Multi-task Generalization Experiment

To assess whether pattern blindness generalizes across algorithmic domains, we extended the L4 evaluation to two additional tasks beyond Fibonacci: **T2 (is_sorted)**: write a function returning True if a list is ascending, and **T3 (count_vowels)**: count vowel characters in a string. Both use the identical shared prompt header (two worked examples with inverted semantics, no explicit rule)—the same prompt structure as Ablation Variant A. We ran $n = 10$ per task for `gpt-4o` and `gpt-4o-mini` (60 total additional runs). Results are in Table V.

H. Reproducibility

All experimental artifacts, commands, and results are committed to the repository under `docs/research/results/`. See reproducibility protocol section for commands.

TABLE II
L1 evaluation: KLR and PSS per model. Baseline rows use $n = 120$; ctx-pack rows for gpt-4.1 family and o4-mini use $n = 20$ (v1–v20 subset). Models sorted by KLR. Context-pack delivery consistently reduces KLR; gpt-4.1-mini ctx-pack achieves the lowest KLR (0.21).

Model	Delivery	KLR↓	PSS↑
gpt-4.1-nano	baseline	0.69	0.16
gpt-5.4-mini	baseline	0.67	0.25
gpt-4.1-mini	baseline	0.43	0.33
gpt-4o-mini	baseline	0.42	0.27
gpt-4.1-nano	ctx-pack	0.38	0.30
gpt-4o	baseline	0.37	0.40
gpt-4.1	ctx-pack	0.30	0.43
o4-mini	ctx-pack	0.27	0.41
gpt-4o-mini	ctx-pack	0.26	0.32
gpt-4.1-mini	ctx-pack	0.21	0.37

TABLE III
L2–L4 outcomes by model and condition. L2 is trivially adopted; L3 reveals a capability threshold; L4 (example-only delivery) defeats all models, including gpt-4o which succeeds at L3.

Model	Level	Delivery	Pass	Failure mode
gpt-4o-mini	L2	Expl. grammar	5/5	None (adopted)
gpt-4o-mini	L3	Expl. rule	0/5	Prior dominance
gpt-4o	L3	Expl. rule	10/10	— (success)
gpt-4.1-mini	L3	Expl. rule	10/10	— (success)
gpt-4o	L4	Examples only	0/5	Pattern blindness

V. Results

A. L1: Partial resistance; scale does not predict compliance

All models show non-zero KLR (0.21–0.69), confirming prior dominance. Three findings stand out: (1) gpt-5.4-mini (KLR=0.67) underperforms gpt-4o (KLR=0.37), showing model scale does not predict compliance; (2) context-pack delivery consistently reduces KLR—gpt-4.1-mini achieves the lowest KLR (0.21 vs. 0.43 baseline, -51%); (3) the reasoning model o4-mini (KLR=0.27) performs comparably to gpt-4o-mini ctx-pack (0.26), suggesting chain-of-thought reasoning provides modest benefit for alias compliance.

B. L2: Syntax inversion not resistant

All models adopted L2 syntax fully (SIR=0.0) when given explicit rules. Structural novelty alone does not create LLM resistance.

C. L3: Capability threshold at semantic rule-following

With an explicit semantic inversion rule, gpt-4o-mini fails completely (PPR=1.0): it generates standard Python despite instruction. gpt-4o and gpt-4.1-mini succeed (PPR=0.0). This reveals a model capacity threshold for following semantic meta-rules.

D. L4: Pattern blindness across all tested models

Without explicit rule, gpt-4o—which succeeds at L3—fails at L4 (PPR=1.0), producing canonical Python and ignoring example-embedded inversion cues. Figure 3 shows the contrast.

TABLE IV

L4 ABLATION (PPR = PYTHON PRIOR RATE). ORIGINAL 4 MODELS: $n = 50$ (5 VARIANTS × 10 RUNS). EXTENDED 3 MODELS: $n = 20$ (5 × 4). PASS RATE = 0 FOR ALL MODELS EXCEPT O4-MINI (1/20). O4-MINI IS THE ONLY MODEL TO ACHIEVE ANY L4 PASS, ON VARIANT E (DIFFERENT-DOMAIN EXAMPLES).

Var.	Description	*gpt-4o*	*gpt-4o-mini*	*gpt-4.1*	*gpt-4.1-mini*	*gpt-4.1-nano*	*gpt-5.4-mini*
A	2ex-baseline	.40	.10	1.0	.10	1.0	1.0
B	3ex-extra	1.0	1.0	1.0	.00	1.0	1.0
C	2ex-no-comment	.90	1.0	1.0	.80	1.0	1.0
D	2ex-swapped	.70	1.0	1.0	.00	1.0	1.0
E	3ex-diff-domain	.50	1.0	1.0	1.0	1.0	1.0†
Overall PPR		.70	.82	1.0	.38	1.0	1.0
Pass rate		0/50	0/50	0/20	0/50	0/20	0/50

†gpt-5.4-mini var-E: 8 HTTP errors; $n_{\text{eff}} = 2$. *o4-mini var-E: 1/4 pass (only

L4 ablation (n=50 per model). We scaled the L4 evaluation across seven models: four at $n = 50$ and three additional models (gpt-4.1, gpt-4.1-nano, o4-mini) at $n = 20$. Results are shown in Table IV. Six of seven models achieve 0% pass rate; the sole exception is `o4-mini` (1/20 pass on variant E), suggesting that chain-of-thought reasoning may provide a narrow pathway to overcoming pattern blindness. PPR values range from 0.38 (gpt-4.1-mini) to 1.0 (gpt-4.1, gpt-4.1-nano, gpt-5.4-mini). Notably, gpt-4.1 and gpt-4.1-nano show PPR=1.0 across *all* variants, indicating complete prior dominance with no evidence of inversion detection. The o4-mini result is particularly interesting: as a reasoning model, it partially overcomes pattern blindness where standard models cannot, though only on the most diverse prompt variant (E, different-domain examples).

E. L4 Multi-task Generalization

To assess whether pattern blindness extends beyond Fibonacci, we evaluated `gpt-4o` and `gpt-4o-mini` on two additional tasks (T2: `is_sorted`; T3: `count_vowels`) using the identical example-only prompt structure ($n = 10$ per task). Results are in Table V.

Three findings emerge: (1) Pattern blindness is *task-dependent* for gpt-4o: it fails on Fibonacci (T1, PPR=1.0) but applies L4 structural inversion on is_sorted (T2, 10/10 pass). We attribute this to the is_sorted task mapping more naturally onto the 'return True/False inside condition' example pattern; (2) gpt-4o-mini shows full prior dominance on T2, producing standard Python 'if lst[i]>lst[i+1]: return False' code; (3) On T3, neither model applies the conditional inversion rule—instead, gpt-4o uses *operational inversion* (`count -= 1; return -count`), a distinct workaround that achieves correct arithmetic while ignoring the semantic inversion rule. This reveals a third failure sub-mode beyond prior dominance and pattern blindness: **operational substitution**, where the model compensates via output transformation rather than rule adoption.

Panel A: L3 — explicit rule (gpt-4o)

System: "if-blocks run when condition is FALSE."

gpt-4o output **(CORRECT):**

```
if n > 1:        % inverted: runs when
FALSE
return n
return fib(n-1) + fib(n-2)
```

Panel B: L4 — examples only (gpt-4o)

User: [2 worked examples with inverted semantics, no explicit rule]

gpt-4o output **(INCORRECT):**

```
if n <= 1:       % Python prior, not
inverted
return n
return fib(n-1) + fib(n-2)
```

Fig. 3. Prior dominance vs. pattern blindness isolated on the same model (gpt-4o). **A**: gpt-4o correctly inverts logic with explicit rule (L3). **B**: The same model reverts to Python prior when the rule is embedded only in examples (L4). This contrast isolates pattern blindness as a failure mode distinct from prior dominance.

TABLE V

L4 MULTI-TASK GENERALIZATION ($n = 10$ PER TASK). T1=FIBONACCI (REPLICATED), T2=IS_SORTED, T3=COUNT_VOWELS. PASS RATE AND PPR (PYTHON PRIOR RATE). `GPT-4O` PASSES T2 USING L4-STRUCTURED CODE; `GPT-4O-MINI` SHOWS FULL PRIOR DOMINANCE ON T2. T3 REVEALS A THIRD FAILURE SUB-MODE: *operational inversion* (MODEL INVERTS THE ACCUMULATION OPERATOR RATHER THAN THE CONTROL-FLOW CONDITION).

Task	Model	Description	Pass	PPR
T1 fib	gpt-4o	fibonacci	0/10	1.00
T1 fib	gpt-4o-mini	fibonacci	0/10	0.00*
T2 sorted	gpt-4o	is_sorted	10/10	0.00
T2 sorted	gpt-4o-mini	is_sorted	0/10	1.00
T3 vowels	gpt-4o	count_vowels	1/10	0.00
T3 vowels	gpt-4o-mini	count_vowels	0/10	0.00

*gpt-4o-mini uses split base cases (`if n==0`/`if n==1`) without `n<=1`; fails judge but i distinct failure sub-mode (*template fragmentation*, not Python-prior revert).

F. Two Failure Modes

- **Prior Dominance** (L1, L3-weak, L4-T2-mini): stated rule is overridden by pretrained token distribution.
- **Pattern Blindness** (L4-T1, L4-T3): model cannot extract semantic rule from examples, even when the same model follows the rule when stated explicitly.
- **Operational Substitution** (L4-T3-gpt-4o): model achieves correct output via arithmetic workaround rather than adopting the semantic inversion rule, indicating awareness of the task goal but not of the L4 language semantics.

The multi-task results refine the pattern-blindness finding: it is not uniform across tasks, but manifests as a interaction between

model capacity, task familiarity, and pattern extractability from examples.

VI. Discussion

A. Effective LLM-resistant language design

Our results identify a principled recipe for LLM-resistant programming languages:

1) Token-level perturbation alone is insufficient (models partially resist, but at measured rates).
2) Syntactic restructuring is also insufficient (models adopt explicit structural rules).
3) Semantic inversion with explicit instruction provides capacity-dependent resistance.
4) Semantic inversion without explanation (L4) provides consistent resistance via pattern blindness.

The most practical target for educational use is L4: retain Python surface syntax, invert conditional semantics, and provide only worked examples rather than explicit rules.

B. Implications for programming education

Our findings directly address the concern of LLM-solved homework. Assignments written in L4-style confusion languages cannot be trivially solved by current LLMs because: (1) the model defaults to Python prior when encountering familiar syntax, and (2) even when examples are provided, pattern extraction of semantic inversion is beyond current few-shot capability.

Students who understand the L4 rules (from lecture instruction) retain a decisive advantage over LLM-assisted submissions, restoring the discriminative power of programming assessments.

C. Limits

While the multi-task experiment (Section V) extends our evaluation beyond Fibonacci, the sample is limited to three tasks and two models ($n = 10$ each). The task-dependent pass rate for `gpt-4o` on T2 (is_sorted: 10/10 pass vs. T1 fib: 0/10) suggests that L4 resistance is not unconditional—tasks whose example patterns closely map to standard Python idioms may be more susceptible to correct inversion. The PPR metric for L3/L4 is task-specific and requires per-task oracle semantics. Human baseline experiments (Exp-5) are needed to confirm that L4 does not also confuse human learners. The newly observed *operational substitution* failure mode (Section V) warrants further characterization: it achieves correct output without adopting L4 semantics, and its detectability may require execution-based rather than structural judgment.

The L4 ablation ($n = 50$ per model) strengthens the pattern blindness finding: zero pass rate holds across all five prompt variants and all four models. However, the high PPR variance across variants (e.g., gpt-4.1-mini variant B PPR=0.0 vs. variant E PPR=1.0) warrants caution—some configurations may trigger different failure sub-modes that are not yet fully characterized. A parallel run of gpt-5.4-mini completed $n = 42$ valid evaluations (8 HTTP errors in variant E); all valid runs show PPR=1.0 and pass=0, consistent with full prior dominance across all variants. The overall PPR of 1.00 for gpt-5.4-mini represents the strongest prior dominance among all tested models.

D. Next steps

- Exp-4 (partial): Multi-task results acquired for T2/T3; broader task coverage (nested control-flow, stateful logic) remains for future work.
- Exp-5: Human study comparing student performance vs. LLM performance on L4 tasks; critical for validating the educational claim.
- Operational substitution characterization: execution-based judge to detect cases where correct output is achieved without L4 rule adoption.
- L4 difficulty scaling: graded version with more/fewer examples, measuring the tipping point at which models begin to extract the inversion pattern.

VII. Conclusion

We presented a four-level taxonomy (L1–L4) for designing LLM-resistant programming languages, grounded in the NTP prior structure of language models.

Our central finding is that resistance does not come from surface-level token changes or structural novelty—both of which LLMs can adapt to with explicit instruction. Robust resistance requires semantic inversion: the same Python-like surface, but inverted conditional semantics. When semantic inversion is delivered implicitly (via examples only), all tested models exhibit pattern blindness and default to Python priors.

We identify two distinct failure mechanisms: *prior dominance* (prior overrides stated rule) and *pattern blindness* (model fails to extract semantic rule from examples). These mechanisms have different implications for language design: prior dominance can be partially mitigated by explicit context packs; pattern blindness requires implicit delivery.

These findings provide a principled basis for designing programming course assessments that remain meaningful in the LLM era, where LLMs will fail on L4-style problems even as they trivially solve standard Python assignments.

VIII. Prompt and Evaluation Protocol

A. L1: Token Alias Substitution

Alias sampling strategy. Representative mappings from three prompt versions:

Token	v1 alias	v35 alias	v90 alias
`if`	`return`	anti-prior phrase	long natural phrase
`def`	`if`	`def`	`return`
`for`	long phrase	`xk`	long phrase
`return`	long phrase	long phrase	`out`
`print`	`do`	—	`qn`

S0 — Baseline system prompt (L1)

"Solve the given prompt exactly. Return ONLY code (no explanation, no markdown). Follow alias mapping strictly."

S1 — Context-pack system prompt (L1, ablation)

"Solve the given prompt exactly. Return ONLY code. Follow alias mapping strictly."

[Transformed language contract]
Use transformed aliases as listed in the prompt. Do NOT output original Python keywords for transformed entries. Output only executable code in the transformed language.

Fig. 4. L1 system prompts. S0 is the compact baseline; S1 adds an explicit alias contract to test whether contextual reinforcement reduces prior dominance.

L1 user prompt template

[Language Description]
Python grammar remains unchanged.
Keyword aliases are user-configurable strings.
At runtime, detected aliases are normalized back to Python.

Current mapping summary:
`if` → {alias} `in` → {alias} `def` → {alias}
`for` → {alias} `return` → {alias} `print` → {alias}

[Task]
1) Define function `fib(n)`.
2) If `n <= 1`, return `n`.
3) Otherwise return `fib(n-1) + fib(n-2)`.
4) Loop `i = 0..6`, print `(i, fib(i))`.

Fig. 5. L1 user prompt template. Alias values are varied across v1–v120, using tokens of differing length, semantic distance, and collision potential. The task is fixed as Fibonacci to control task difficulty.

L2 grammar specification (complete)

This is NOT Python. All Python keywords are FORBIDDEN.

`:define name [args] ->` (replaces `def name(args):`)
`<= value` (replaces `return value`)
`value ?cond: res ;else: alt ;end` (replaces `if`)
`loop var over iterable { body }` (replaces `for`)
`;;` closes function block

Fig. 6. L2 grammar specification as given to the model. Models must produce code using exclusively these constructs. Result: all models fully adopted the syntax (SIR=0.0), showing LLMs readily follow explicit structural rules.

B. L2: Syntax Inversion

C. L3: Semantic Inversion (Explicit)

D. L4: Semantic Inversion (Implicit)

E. Evaluation Metrics

- **KLR** (Keyword Leakage Rate): fraction of alias slots where original Python keyword was used. ACR $= 1-$KLR is a derived quantity.
- **PSS** (Partial Structural Score): weighted sub-component credit (alias: 0.4, parse: 0.25, skeleton: 0.25, format: 0.1).
- **PPR** (Python Prior Rate, L3/L4): fraction of outputs using Python-style condition without inversion.

F. Judge Protocol (L1)

1) Alias compliance: check that alias is present and original token is absent.

L3 system prompt — explicit semantic rule

*"Follow the L3 language rules exactly. If-blocks run when condition is **FALSE**."*

L3 user prompt (complete)

L3 rule: Syntax is identical to Python, but:
RULE: Indented block executes when condition is **FALSE**.
RULE: Else-block executes when condition is **TRUE**.

Example:

```
if x > 5:
    print("small")  # runs when NOT x > 5
else:
    print("large")  # runs when x > 5
```

Task: Write `fib(n)` in L3. Intended logic: if $n \leq 1$ return n; else return `fib(n-1)+fib(n-2)`.
Think carefully: your if-block runs when condition is FALSE.

Fig. 7. L3 prompts. The rule is stated explicitly in the system message. Weaker models (gpt-4o-mini) still fail (PPR=1.0), while stronger models succeed—revealing a capability threshold for semantic meta-rule following.

L4 user prompt (example-only delivery, no explicit rule)

Here are some examples in our programming language:

Example 1 — max function:

```
def max_val(a, b):
    if a > b:
        return b   % smaller when a > b
    return a
```

Output: `max_val(3,5)=3`, `max_val(7,2)=2`

Example 2 — even check:

```
def is_even(n):
    if n % 2 == 0:
        return False
    return True
```

Output: `is_even(4)=False`, `is_even(3)=True`

Task: Write `fib(n)` in this language. Target: `fib(0)=0`, `fib(1)=1`, `fib(n)=fib(n-1)+fib(n-2)`.
Print `fib(0)` through `fib(6)`.

Fig. 8. L4 prompt (no system message). The semantic inversion rule is not stated; models must infer it from worked examples. Even gpt-4o, which succeeds at L3, defaults to canonical Python fib—exhibiting *pattern blindness*.

2) Normalization: replace aliases → Python keywords via word-boundary regex.
3) Parse check: `ast.parse()` on normalized code.
4) Skeleton check: verify `fib`, base case, recursion, and print loop.

G. Judge Protocol (L3/L4)

- **Python-prior**: model writes `if n <= 1: return n` (non-inverted) ⇒ PPR=1 (fail).
- **L3-correct**: model writes `if n > 1: return n` or equivalent ⇒ PPR=0 (pass).

IX. Complete Experiment Log

A. All experiments (chronological)

1) **E1–E4: L1 baseline, $n = 120$, 4 models (2026-03-23).** Models: gpt-4o-mini, gpt-4.1-mini, gpt-4.1-nano, gpt-5.4-mini. Files: `batch-v1-120.`*`model`*`.json`
2) **E5: L1 context-pack ablation, $n = 120$, gpt-4o-mini (2026-03-23).** Setting: CONTEXT_PACK=1. KLR: $0.423 \rightarrow 0.260$ (39% reduction from E1).
3) **E6: L1 baseline, $n = 120$, gpt-4o (2026-03-24).** KLR=0.366, highest structural compliance.
4) **Partial judge applied (2026-03-24).** Script: `partial_judge.py` applied to all L1 batches.
5) **E7: L2 syntax inversion, $n = 5$, gpt-4o-mini (2026-03-24).** Explicit grammar rules provided in system prompt. SIR=0.0; model adopted L2 syntax fully.
6) **E8: L3 semantic inversion, $n = 5$, gpt-4o-mini (2026-03-24).** Explicit inversion rule in system prompt. PPR=1.0: prior dominance, complete failure.
7) **E9–E10: L3 semantic inversion, $n = 10$, gpt-4o and gpt-4.1-mini (2026-03-24).** Both models: PPR=0.0 (successfully followed rule).
8) **E11: L4 implicit semantic, $n = 5$, gpt-4o (2026-03-24).** No system prompt; only 2 worked examples shown. Pattern blindness observed: model ignored inversion cue.

B. API and runtime settings

Setting	Value
Temperature	0
Base URL	`https://api.openai.com/v1`
Timeout	120s
Prompt delay	0.4s
API key source	AVRTG project env
Retry	None

C. Repository structure for replication

Script / Path	Purpose
`run_gpt54_eval.py`	L1 batch runner
`run_l2_eval.py`	L2 batch runner
`partial_judge.py`	Continuous metrics
`compare_eval_runs.py`	Run comparison
`docs/archive/*/`	Prompt banks (L1–L4)
`docs/research/results/`	All output JSONs
`language-levels/`	Level specs

X. Appendix: Prompt Samples, Taxonomy, and Config Snapshot

A. Prompt samples (readable form)

Representative prompt-map blocks from the versioned bank:

Sample A (v1)

- if → return
- def → if
- for → long natural phrase (renamed symbol)
- in → long natural phrase (strict compilation token)
- return → long natural phrase (keyword mapping token)
- print → do

Sample B (v35)

- in → mapped token phrase
- def → def
- for → xk
- return → long phrase preserving grammar
- if → anti-prior phrase

Sample C (v90)

- def → return
- for → phrase mapped back to reserved token
- in → exact mapping phrase
- return → out
- if → long natural phrase
- print → qn

B. Violation taxonomy summary (gpt-4o-mini, v1–v120)

From the context-pack comparison artifact, dominant violation categories are:

- Original keyword leakage (e.g., `for`, `if`, `return`, `in`, `def`)
- Parse failure after normalization (line-specific syntax failure)
- Missing alias emissions for required transformed tokens

Top counts in context-pack run include:

- `for` leaked: 52
- parse failed (line 1): 35
- `if` leaked: 33

C. Execution config snapshot (non-secret)

TABLE VI
KEY RUNTIME SETTINGS

Setting	Value
Models	gpt-4o, 4o/4.1-mini, 4.1-nano, 5.4-mini
Range	v1–v120
CONTEXT_PACK	0/1
Temperature	0
Judge	deterministic

Secret-bearing values (API keys) are excluded and loaded via runtime environment. Language specs are in `language-levels/` (repository).

XI. Reproducibility and Artifact Availability

All experiments are committed to the repository. Representative paths:

- Results: `docs/research/results/`
- Language specs: `language-levels/`
- Evaluation scripts: `scripts/partial_judge.py`, `scripts/run_l2_eval.py`
- Paper: `paper/overleaf/`

References

[1] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] J. Austin *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[3] T. Zhuo *et al.*, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," arXiv:2406.15079, 2024.

[4] N. Jain *et al.*, "LiveCodeBench: Holistic and contamination-free evaluation of large language models for code," arXiv:2403.07974, 2024.

[5] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *EMNLP-Findings*, 2020.

[6] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models," in *EMNLP*, 2021.

[7] D. Fried *et al.*, "InCoder: A generative model for code infilling and synthesis," 2022.

[8] Z. Luo *et al.*, "WizardCoder: Empowering code large language models with evol-instruct," arXiv:2306.08568, 2024.

[9] D. Guo *et al.*, "DeepSeek-Coder: When the large language model meets programming," arXiv:2401.14196, 2024.

[10] Y. Liu *et al.*, "CodeIF: Benchmarking instruction-following in code generation," arXiv:2502.19166, 2025.

[11] W. Zhang *et al.*, "MultiCodeIF: Comprehensive evaluation of code instruction-following," arXiv:2507.00699, 2025.

[12] E. Chen *et al.*, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," arXiv:2404.13208, 2024.

[13] J. Xie *et al.*, "Adaptive chameleon or stubborn sloth: Revealing the behavior of LLMs in knowledge conflicts," ICLR 2024 (arXiv:2305.13300), 2024.

[14] S. Wu *et al.*, "Untangle the KNOT: Interweaving conflicting knowledge and reasoning skills in LLMs," arXiv:2403.08319, 2024.

[15] Z. Zhang *et al.*, "Knowledge conflicts for LLMs: A survey," EMNLP 2024 (arXiv:2403.08319), 2024.

[16] F. Shi *et al.*, "Large language models can be easily distracted by irrelevant context," ICML 2023 (arXiv:2302.00093), 2023.

[17] Z. Wang *et al.*, "A law of next-token prediction in large language models," arXiv:2408.13442, 2024.

[18] S. Min *et al.*, "Rethinking the role of demonstrations: What makes in-context learning work?" EMNLP 2022 (arXiv:2202.12837), 2022.

[19] Q. Dong *et al.*, "A survey on in-context learning," arXiv:2301.00234, 2023.

[20] A. Lampinen *et al.*, "A-Not-B errors in LLMs: Limitations of in-context learning," arXiv:2409.15454, 2024.

[21] Q. Zhao *et al.*, "LLMs struggle to perform counterfactual reasoning with parametric knowledge," arXiv:2410.06392, 2024.

[22] M. Turpin *et al.*, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," NeurIPS 2023 (arXiv:2305.04388), 2023.

[23] K. Valmeekam *et al.*, "LLMs still can't plan; can LRMs? evaluation of OpenAI's o1," arXiv:2409.13373, 2024.

[24] K. Zhu *et al.*, "PromptBench: Evaluating robustness of large language models," arXiv:2306.04528, 2023.

[25] R. Pan *et al.*, "Attacking LLMs with code generation: Semantically-aligned adversarial perturbations," arXiv:2412.16359, 2024.

[26] S. Jha *et al.*, "Break the chain: Reasoning failures in LLMs via adversarial code prompting," arXiv:2506.06971, 2024.

[27] A. Khan *et al.*, "Defeating LLMs with adversarial perturbations in programming assignments," arXiv:2410.09318, 2024.

[28] J. Wu *et al.*, "A survey on LLM-based code generation for low-resource and domain-specific languages," arXiv:2410.03981, 2024.

[29] W. Chen *et al.*, "ConCodeEval: Evaluating LLMs for code constraint comprehension in DSLs," EMNLP 2024, 2024.

[30] A. Ni *et al.*, "L2CEval: Evaluating language-to-code generation capabilities," TACL 2024, 2024.

[31] Y. Dong *et al.*, "XGrammar: Flexible and efficient structured generation for LLMs," arXiv:2411.15100, 2024.

[32] J. Muendler *et al.*, "Type-constrained code generation with language models," PLDI 2025, 2025.

[33] C. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" 2024.

[34] A. Dakhel *et al.*, "GitHub Copilot AI pair programmer: Asset or liability?" 2023.

[35] S. Lau and P. Guo, "From coding assistant to software developer: How novice programmers use LLMs," arXiv:2304.01984, 2023.

[36] J. Prather *et al.*, "It's complicated: Perspectives of novice programmers on GitHub Copilot," arXiv:2303.08733, 2023.

[37] M. Liffiton *et al.*, "CodeHelp: Using large language models with guardrails for programming classes," arXiv:2308.06921, 2023.

[38] E. Novak and B. McDanel, "Designing LLM-resistant programming assignments," SIGCSE TS 2025, 2025.

[39] M. Orenstrakh *et al.*, "Detecting LLM-generated code assignments using perplexity of code," arXiv:2307.09714, 2023.

[40] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," NeurIPS 2022 (arXiv:2203.02155), 2022.